

CST8221 - Java Application Programming

ASSIGNMENT 2 – Basic Interface (QMillionaire¹)

General View

Due Date: prior or in **June 7 , 2026 (midnight)**

- **2nd Due date** (until June 14th 2026) - **50%** off.
- **NOTE:** Submissions must be done by Sunday midnight.

Earnings: **8%** of your course grade.

Purpose: Create the interface and basic layout for the **QMillionaire (Quantum Game)**.

- ❖ This is the second task in JAP. Now, you are moving to implementation of the **WHTBM quantum version** application. Note that this project will be updated to incorporate new functionalities in upcoming activities.



- Game:
 - <https://www.arkadium.com/games/millionaire/>
 - <https://wwbm.com>
 - <https://www.mindgames.com/game/Millionaire>

- ❖ **Note.** In the original version of WWBM (see links above), questions at an increasing difficult level are posted and that one that can answer all of them correctly, can reach the prize of 1M. The game's mechanics uses a timer, and a set of help options for the candidate. It is exactly what will be different in the quantum version (to be explained only later).

- ❖ **PART I:** What you need to **create the interface** considering the model:
 - The main window for the application (**QMillionaire**) must have two basic modes:

¹ Check the A1 specification for previous details.

- **Design mode:** Where basic functionalities for future database records will be defined.
- **Play mode:** The code for the game itself, where one specific player must start the quiz.
 - A basic controller (to be increased in the next assignments).
- ❖ **PART II:** Additional elements are required to define:
 - **Code** (using code conventions).
 - **Javadoc** files (created by documental comments in the code).
 - **Executable Jar** (remember to be sure that it is not a common archive file).

Note 1: About Teams

As mentioned, new teams are acceptable up to the end of for assignment. It means that after this period, if you decide to work alone, but this is the **last opportunity to compose teams** – starting with A3, no **more teams** will be possible to be composed.

Part I – Creating the Game

1.1. Basic Interface

The **QMillionaire** has a simple visual in which you need to be able to simulate the game.



Fig.1 – QMillionaire main screen proposal

❖ Notes:

- The proper interface can vary (as well as the proposed components – labels, buttons, etc.).

- The purpose of this assignment is just to create an interface (no game action is required), but minimal configurations should be able to be set:
 - Language settings (Internationalization).
 - Layout (look and feel).

The player interface is the main interface and must have the minimal requirements to simulate the WWTBM game, using a different version (based on quantum rules) for assistance / help. See the proposal as follows.

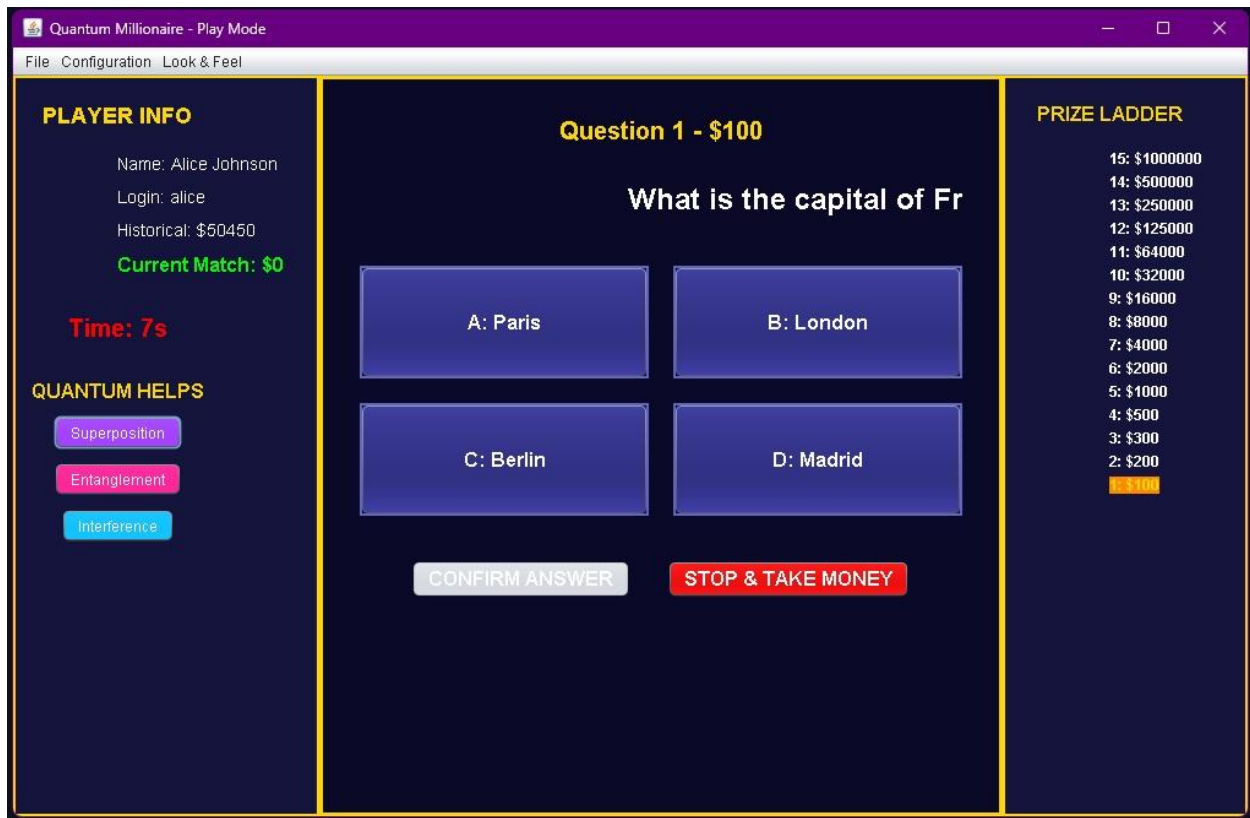


Fig.3 – QMillionaire quiz [rp[psa;

- In the second (and most important interface), the **player** user must be able to **play** the game – basically answering the quiz questions / requesting help or stopping the game execution.
- ❖ The application must be able to play the game, but at the same time, change some properties:
 - Basic configurations
 - Change the interface (Look and Feel).
 - Change the language (English and another to be freely defined).
 - Change colors.


```
mkdir %BIN_DIR%
REM Compile Java source files
javac -d %BIN_DIR% %SRC_DIR%\qmillionaire\*.java

echo 'Creating Jar ..... '
REM Copy resources and images to bin
echo Copying resources and images...
xcopy /s /y %RES_DIR% %BIN_DIR%\resources\
xcopy /s /y %IMG_DIR% %BIN_DIR%\images\
REM Create manifest file
echo Creating manifest...
echo Main-Class: %MAIN_CLASS% > manifest.txt
cd %BIN_DIR%
REM Create JAR file
jar cfm ../JAR_NAME% ../manifest.txt *
cd ..
REM Clean up manifest
del manifest.txt

echo 'Running Jar ..... '
java -jar %JAR_NAME%

echo 'End ..... '
```

2.1. BASIC DOCUMENTATION

CODE ELEMENTS (*Basic criteria*)

Code conventions ideas:

- Code well organized, methods and functions grouped logically, classes in order, etc.
- Label naming (variables, constants, methods, functions, etc.) consistent and meaningful.
- Code consistently indented and spaced (define a consistency /organization).
- Good commenting which includes block comments.

JAVADOC (*following Java Standard*)

- Javadoc outputs are required, and the pages / folders must be included.

JAR (*Executable file*)

- Finally, it is required to create and include JAR files (that can open the binary using Java)

Note 3: Read Assignment Standard

The code and patterns to be used in submission must follow the Java Code Conventions, but also the elements described in the Assignment Standard.

Evaluation

- ❖ Please read the Assignment [Submission Standard and Marking Guide](#).

- ❖ **About Plagiarism:** Your code must observe the configuration required. Similarly, we need to observe the policy against ethical conduct, avoiding problems with the 3-strike policy.

Marking Rubric

Maximum Deduction (%)	Deduction Event
	Severe Errors:
100	Late submission (after 1 week due date)
up to 100	Does not compile with java
up to 100	Compile but crashes after launch
up to 100	Does not comply with the problem specifications
up to 50	The GUI appearance is not compliant with functionality.
up to 30	Invalid containers, components, layout managers – talk to professor
up to 10	Inadequate comments (headers, code explanation), insufficient javadoc
up to 30	Unable to run jar by script (mandatory to run outside IDE)
up to 30	Missing component (various text fields, buttons, labels, graphics)
up to 10	Other minor errors
up to 10	Bonus: clean code, elegant code, enhancements, discretionary points (1)
Final Mark	Formula: $10 * ((100 - \sum \text{penalties} + \text{bonus}) / 100)$, max score 11% .

Submission Details

- ❖ **Digital Submission:** Compress into a **zip** file with all files (including source code, images, Javadoc, Jar).
- ❖ Upload the zip file on Brightspace. The file must be submitted prior or on the due date as indicated in the assignment.
- ❖ **IMPORTANT NOTE:** The name of the file must be **Your Last Name** followed by the last three digits of your student number followed by your lab **section number**. For example: **A2_Mwangi234.zip**.
 - If you are working in teams, please, include also your partner. For instance, something like: **A12_Mwangi234_Sousa456.zip**.
- ❖ **IMPORTANT NOTE:** Assignments will not be marked if there are not source files in the digital submission. Assignments could be late, but the lateness will negatively affect your mark: see the **Course Outline** and the **Marking Guide**. All assignments must be successfully completed to receive credit for the course, even if the assignments are late.

File update: May 26, 2026.

Good luck with A2!